笔记详情页 Markdown 排版规则规范-420

⛳ 基于 HyperOS 设计系统，适用于移动端笔记详情页的 Markdown 渲染规范。

一份「设计 + 研发双端可用的完整规范文档」

- ✅ 语义层（Semantic）
- ✅ Scale 体系（可扩展）
- ✅ 响应式规则
- ✅ 暗黑模式
- ✅ Markdown 映射（AI 可用）

你可以把这份直接当成**团队规范 / 设计系统文档 v2.0**

📘 Markdown 排版设计系统规范（AI Native ）

一句话目标：

让 Markdown 成为「设计 → 研发 → AI」的统一中间协议

一、整体架构（核心升级）

Semantic Layer（语义层）

↓

Typography Layer（字体系统）

↓

Layout Layer（间距系统）

↓

Component Mapping（组件映射）

↓

Markdown Mapping（AI/文档协议）

二、Semantic Token（语义层）

1. 语义定义

分类	名称	Token	含义	使用场景
页面标题	页面标题 (H0)	heading.page	页面标题	页面唯一主标题
	一级标题 (H1)	heading.section	模块标题	一级模块
	二级标题 (H2)	heading.subsection	子模块	二级内容
	三级标题 (H3)	heading.caption	辅助标题	说明/补充
基础文本	正文	text.primary	主文本	正文
	注释/辅助	text.secondary	次级文本	注释/辅助
样式-强调	加粗	text.emphasis	强调	重点信息
	行内代码	text.code	代码/术语	技术内容
区块	引用	block.quote	引用	引用内容
	有序/无序列表	block.list	列表	信息结构
	分割线	block.divider	分割	内容分区

---

2. 关键原则

- ❌ 不允许直接写“16dp”
- ✅ 必须写语义（如 `text.primary`）

三、Typography Token（字体系统）

1. 字号 & 字重体系（统一 scale）

语义	字号	字重	行高倍数
heading.page	26dp	Semibold-520	1.3x
heading.section	22dp	Demibold-450	1.35x
heading.subsection	18dp	Demibold-450	1.45x
heading.caption	16dp	Demibold-450	1.5x
text.primary	16dp	Medium-380	1.6x
text.secondary	13dp	Regular-330	1.2x
text.code	14dp	Regular-330	1.2x

---

2. 统一计算规则（工程必须用）

标题行高 = fontSize × 1. ~ 1.4

正文行高 = fontSize × 1.6

代码/辅助信息 = fontSize × 1.2

行高 (Line Height) 的计算逻辑

行高通常由 **“字号 × 倍数”** 决定。在移动端 Markdown 场景下，由于屏幕较窄，视线换行频率高，行高需要比桌面端略大。

公式：L = F x M

- L：行高 (Line Height)
- F：当前字号 (Font Size)
- M：行高倍数 (Multiplier)

倍数 (M) 的取值建议：

- **正文 (Body)**：建议 M在 **1.6**
  - *逻辑*：若字号为 16dp，行高应为 16 x1.6 约等于26pt。这是移动端长文阅读的“黄金比例”。
- **标题 (Headings)**：建议 M 在 **1 ~ 1.4** 之间。
  - *逻辑*：标题字号大，若倍数过大会导致视觉松散。例如 22dp 的 H1，22 x1.35 约等于 30dp
- **代码/辅助信息**：建议 M 为 **1.0 ~ 1.2**。

---

3. 字体规范

字体名称	类型	版本
MiSans	简体中文	Version 4.009
MiSans Latin	英文	Version 4.007

四、Layout Token（间距系统）

1. 统一 8dp 体系

Token	数值
xs	4
sm	8
md	12
lg	16
xl	24
xxl	32

---

2. 语义间距规则

场景	间距
段落间距	md (12)
模块间距	xl (24)
标题下间距	sm–md
列表项间距	sm (8)
代码块 padding	md (12)

---

3. 缩进规则（统一）

list indent = 16dp

每级递增 = +16dp

最大层级 = 3

五、Color Token（支持暗黑模式）

1. Light 模式

模式	名称	语义	颜色色值
Light	页面标题 (H0)	heading.page	#000000
	一级标题 (H1)	heading.section	#000000
	二级标题 (H2)	heading.subsection	#000000
	三级标题 (H3)	heading.caption	#000000
	正文	text.primary	#000000 (Alpha 90%)
	注释/辅助	text.secondary	#000000 (Alpha 40%)
	行内代码	text.code	#000000 (Alpha 60%)
	代码块背景	code.bg	#000000 (Alpha 4%)
	分割线	block.divider	#000000 (Alpha 10%)
	引用	block.quote	#000000 (Alpha 60%)

---

2. Dark 模式（新增）

模式	名称	语义	颜色色值
dark	页面标题 (H0)	heading.page	#FFFFFF
	一级标题 (H1)	heading.section	#FFFFFF
	二级标题 (H2)	heading.subsection	#FFFFFF
	三级标题 (H3)	heading.caption	#FFFFFF
	正文	text.primary	#FFFFFF (Alpha 95%)
	注释/辅助	text.secondary	#FFFFFF (Alpha 40%)
	行内代码	text.code	#FFFFFF (Alpha 60%)
	代码块背景	code.bg	#FFFFFF (Alpha 4%)
	分割线	block.divider	#FFFFFF (Alpha 10%)
	引用	block.quote	#FFFFFF (Alpha 60%)

六、Component Mapping（组件映射）

📊 Markdown × UI组件 × Token 映射表（标准版）

名称	Markdown 语法	UI 组件	Design Token (语义层)
页面标题 (H0)	# (一级占位)	PageHeader	heading.page
一级标题 (H1)	#	Header1	heading.section
二级标题 (H2)	##	Header2	heading.subsection
三级标题 (H3)	###	Header3	heading.caption
正文 (Body)	(无)	TextView	text.primary
辅助 (注释)	[^1] 或特定语法	Caption	text.secondary
高亮 (Mark)	==text==	MarkerView	text.highlight
行内代码（Code）			text.code
加粗 (Bold)	**text**	StrongText	text.emphasis
斜体 (Italic)	*text*	EmphasisText	text.italic
下划线 (Underline)	<u>text</u>	UnderlineText	text.link
删除线 (Strike)	~~text~~	StrikeText	text.strike
引用文字 (Quote)	>	Blockquote	block.quote
无序列表 (Unordered)	- 或 *	UnorderedList	block.list.unordered
有序列表 (Ordered)	1	OrderedList	block.list.ordered
分割线 (Divider)	---	HorizontalRule	block.divider
向右缩进 (Indent)	Tab / Space	IndentationGroup	layout.indent
向左缩进 (Outdent)	Shift+Tab	IndentationGroup	layout.outdent
居左对齐 (Left)		TextAlignLeft	layout.align.left
居中对齐 (Center)		TextAlignCenter	layout.align.center
居右对齐 (Right)		TextAlignRight	layout.align.right

组件细节补充

1. Quote

属性	值
左边框	4px
padding	12px
背景	code.bg

---

2. Code Block

属性	值
字体	monospace
背景	code.bg
padding	12
圆角	8

---

3. List

属性	值
行高	跟随正文
间距	8px
多行对齐	顶对齐

七、Markdown 映射规范（AI关键）

1. 标准映射

→ heading.page

→ heading.section

→ heading.subsection

正文 → text.primary

`code` → text.code

- → block.list

> → block.quote

--- → block.divider

---

2. 强调规则

** → text.emphasis

- →（不推荐中文使用）

---

3. 生成约束：

- 每段 ≤ 4 行
- 每段 ≤ 2 个强调
- 列表 ≤ 3 层
- 禁止跳级标题

一套可落地的完整 JSON Schema

AI Native UI System Specification (v1.0)

1. 设计规范（Design Specification）

本系统基于语义优先原则构建，统一设计与研发语言。

结构层

- Heading（page / section / subsection / caption）
- Text（primary / secondary / emphasis）
- Block（list / quote / divider）
- Layout（align / indent）

排版规则

- 正文行高：1.6
- 标题行高：1.3\~1.4
- 段落最大行数：4行
- 列表最大层级：3层

---

2. Token 系统（Design Token System）

Token 分类

Text

- text.primary
- text.secondary
- text.emphasis

Background

- bg.base
- bg.code

Divider

- divider

Theme 支持

```` ``` ```` ```` ```
````

```
    "text.primary": "#111",
    "bg.base": "#fff"
  },
  "dark": {
    "text.primary": "#fff",
    "bg.base": "#000"
  }
}
```

---

3. JSON Schema（结构协议）

Root

```json
{
  "type": "document",
  "theme": "auto",
  "children": []
}
```

Node 基本结构

```json
{
  "type": "paragraph",
  "props": {},
  "children": []
```

}
```

### 支持类型

- heading
- paragraph
- text
- list
- list_item
- quote
- code_block
- divider
- container

---

## 4. TypeScript 类型系统（强约束）

### 核心类型

```ts
export type Node =
  | HeadingNode
  | ParagraphNode
  | ListNode
  | QuoteNode
  | CodeBlockNode
  | DividerNode;
```
```

示例

```ts
export interface HeadingNode {
  type: 'heading';
  props: {
    level: 'page' | 'section' | 'subsection';
  };
  children: TextNode[];
}
```

---

5. 系统能力总结

渲染链路

Markdown → JSON → UI

设计链路

Figma → Token → UI

AI链路

Prompt → Markdown → JSON → UI

---

6. 结论

该系统实现：

- 设计与研发统一语言
- AI可生成 UI
- 支持多端渲染
- 支持主题与暗黑模式

本质为：

UI 描述协议 + 渲染引擎系统